Assignment 2: Unconstrained & Equality Constrained Optimization

Exercise 1:

The left wall must have the same height as the right-side wall. Therefore, $l = \sin(\theta) \cdot a$ (constraint 1). We're also given that all the metal must be 3, so we derive the identity $l + b + a = 3$ (constraint 2).

Substituting (1) into (2), we get $a(\sin(\theta) + 1) + b = 3$. Solving for b, we get that $b = 3 - a - a\sin(\theta)$.

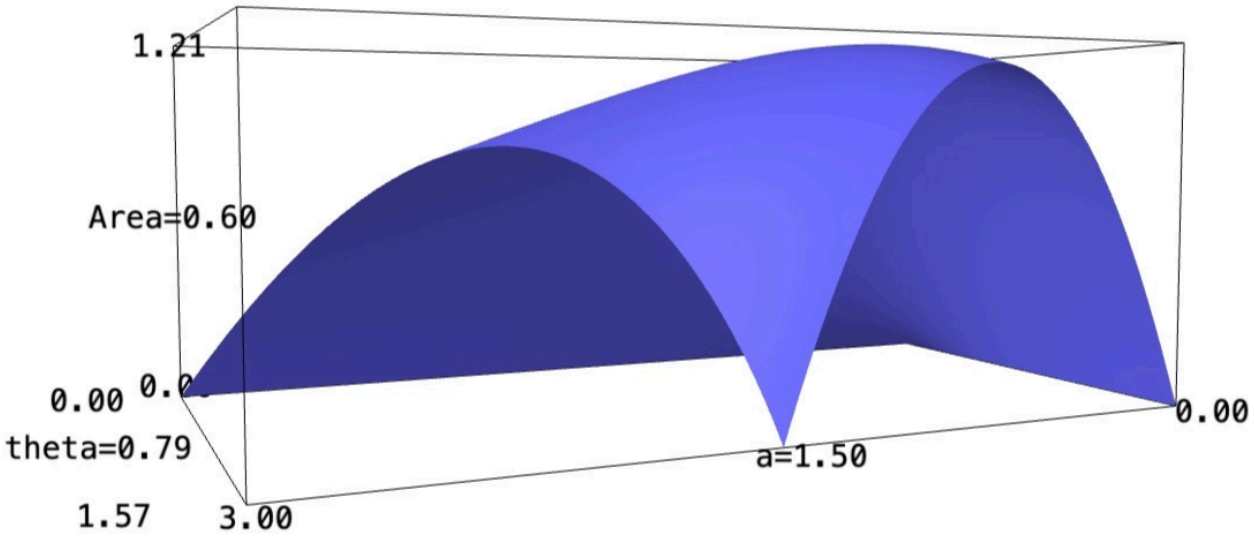
To calculate the area, we have Part 1: $l \cdot b = ab\sin(\theta)$ (the area of the rectangle on the left) and Part 2: $\frac{\cos(\theta) \cdot al}{2}$ (the are of the triangle remaining).

Then, our objective function (area) becomes adding parts 1 and 2 (besides writing b in terms of a and θ), which gives us:

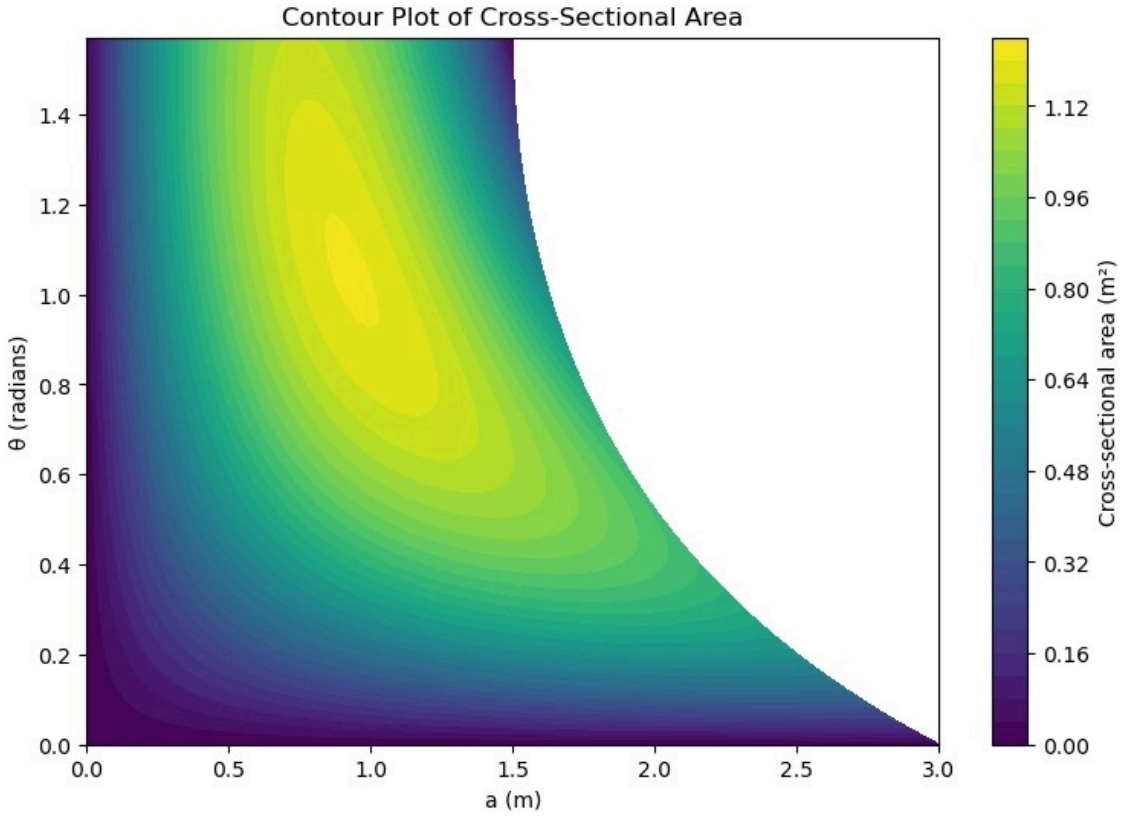
$$A(a, \theta) = a \cdot (3 - a - a\sin(\theta)) \cdot \sin(\theta) + \frac{a^2 \cdot \sin(\theta) \cdot \cos(\theta)}{2}$$

a must be $0 < a < 3$ because the max length of metal that we have is $3m$. However, the bottom value (b) must also be greater than 0. Plugging in this inequality as well, we get that a must lie within $\boxed{0 < a < \frac{3}{1 + \sin(\theta)}}$ Additionally, θ must be between 0° and 90° (or $\frac{\pi}{2}$). For simplicity, we'll consider all of these boundaries inclusive ($\leq$ or $\geq$) and later consider if they make sense if the global maximum happens to lie exactly on them.

Surface Plot:



Contour Plot:



Here, the maximum appears to be at $a \approx 0.9$ m and $\theta \approx 1.05$ rad. It appears to be unique because (1) the surface plot only shows one peak and (2) the contour plot has only one yellow region, which corresponds to one peak.

Exercise 2:

I decided to maximize $A(a, \theta)$ directly, subject to (1) $0 < a < \frac{3}{1+\sin(\theta)}$ and (2) $0 < \theta < \frac{\pi}{2}$:

Taking the partial derivatives, we get:

$\frac{\partial A}{\partial a} = -2a\sin(\theta)^2 + \sin(\theta)(a\cos(\theta) - 2a + 3)$

$\frac{\partial A}{\partial \theta} = a^2\cos(\theta)^2 - 2a^2\cos(\theta)\sin(\theta) - \frac{1}{2}a^2 - (a^2 - 3a)\cos(\theta)$

Then, to find critical points, we create a system such that $\frac{\partial A}{\partial a} = \frac{\partial A}{\partial \theta} = 0$ and solve for (a^*, θ^*). Using SageMath's $\text{find_root}()$ method, I got $(a^*, \theta^*) \approx (0.92820, 1.04719)$.

Now, to prove that this constitutes the global minimum, we can: (1) Apply the 2nd Derivative test (checking for negative definiteness of the Hessian matrix of A, since we're dealing with a multivariable function), and (2) Check against the boundaries that satisfy the constraints of the problem.

Constructing the Hessian Matrix of A: $H_A = \begin{bmatrix} A_{aa} & A_{a\theta} \\ A_{\theta a} & A_{\theta\theta} \end{bmatrix}$. Calculating each one of the entries, we get that:

$A_{aa} = \sin(\theta)(\cos(\theta) - 2 - 2\sin(\theta))$,

$A_{a\theta} = A_{\theta a} = 3\cos(\theta) - 2a\cos(\theta) - 2a\sin(2\theta) + a\cos(2\theta)$,

$A_{\theta\theta} = -3a\sin(\theta) + a^2\sin(\theta) - a^2\sin(2\theta) - 2a^2\cos(2\theta)$.

Plugging in the values for the critical points that we found earlier, we get that $H_A(a^*, \theta^*) = \begin{bmatrix} -2.79904 & -1.5 \\ -1.5 & -1.54998 \end{bmatrix}$.

Calculating the Leading Principal Minors of H_A: 1st Leading Principal Minor: $-2.79904 < 0$, 2nd Leading Principal Minor: $a \cdot c - b^2 \approx 2.08846 > 0$.

From these, we can confirm that $A(a, \theta)$ is negative definite (if we repeat the calculation for $-A$, we get a positive definite Hessian matrix), meaning that (a^*, θ^*) is a maximum. Now, comparing agains the boundaries:

Boundary 1: along $a = 0 \Rightarrow A(0, \theta) = 0$, Boundary 2: along $\theta = 0 \Rightarrow A(a, 0) = 0$

Boundary 3: along $\theta = \frac{\pi}{2}$: Then, $A(a, \frac{\pi}{2}) = 3a - 2a^2$. Solving for critical points, $\frac{dA}{da} = -4a + 3 = 0 \Rightarrow a = \frac{3}{4}$ and $A(\frac{3}{4}, \frac{\pi}{2}) = 1.125$ (we know it's a max because

2nd derivative is positive)

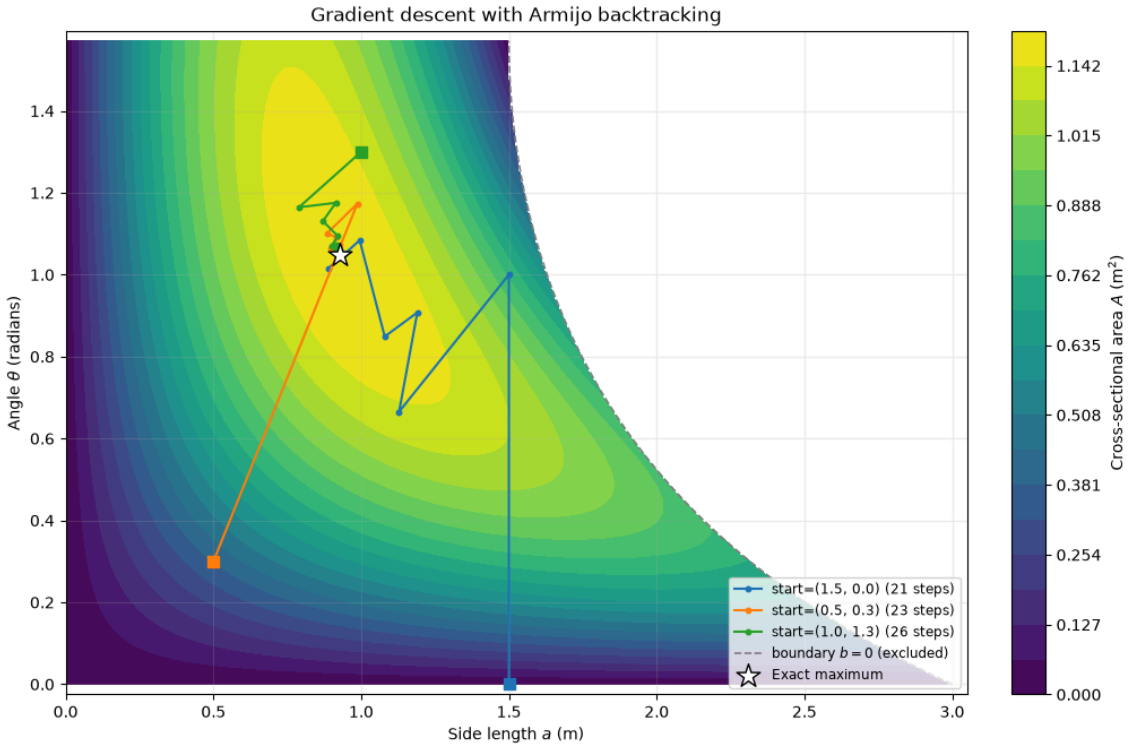
Boundary 4: along $a = \frac{3}{1+\sin(\theta)}$: Then, $A(\theta) = \frac{9\sin(\theta)\cos(\theta)}{2(1+\sin(\theta))^2}$. Differentiating it and solving for $\frac{dA}{da} = 0$, we get that $\theta = \pi/6$, so $A(\theta = \pi/6) = \frac{\sqrt{3}}{2}$ (being a max or min doesn't matter here because $\frac{\sqrt{3}}{2} < 1.125$.

Comparing all the critical points along the boundaries with our previous critical point (a^*, θ^*), we get that the max of $A(a, \theta)$ is $A(a^*, \theta^*) \approx 1.206$ for $(a^*, \theta^*) = (4\sqrt{3} - 6, \frac{\pi}{3})$.

Substituting this back into our expression for b, we get that $b^* = 3 - \sqrt{3} \approx 1.26795m$

Gradient Descent

Chosen starting points: $(1.5, 0), (0.5, 0.3), (1.0, 1.3)$

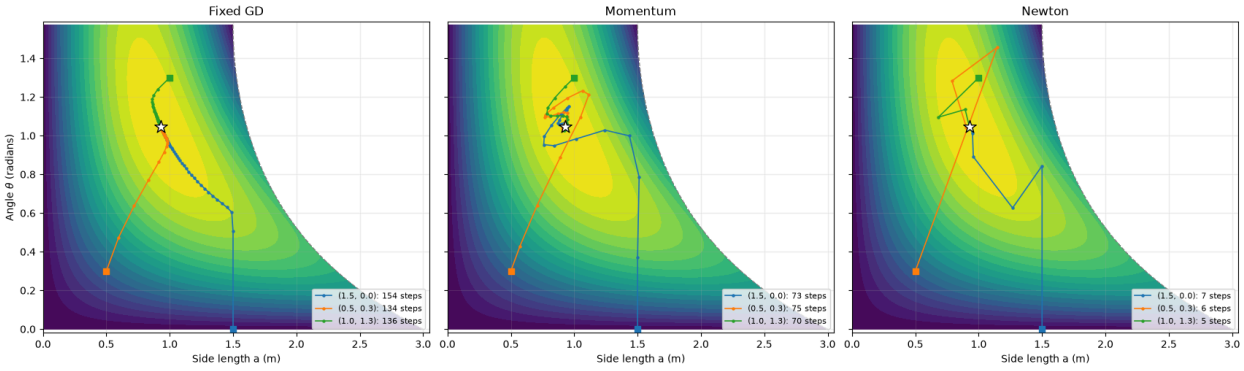


Here, all of the starting points converged. However, contrary to my initial expectations, the closest initial point to the maximum $(1.0, 1.3)$ took the highest number of steps to converge to the maximum (26).

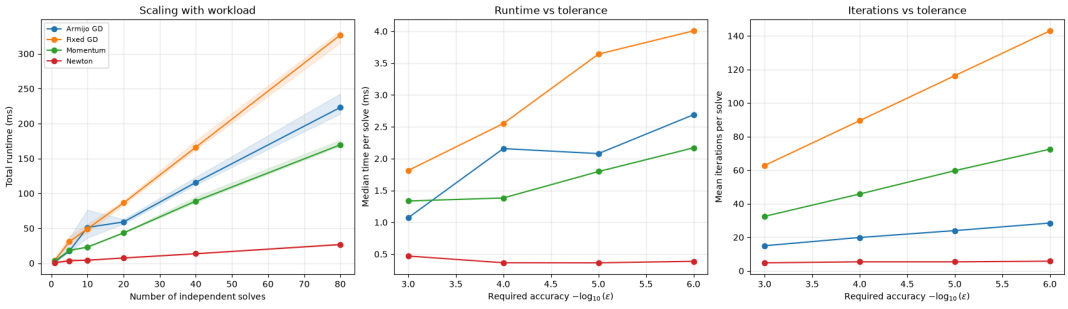
Truncated Table of Steps

method	start	status	iterations	final_area	position_error
Armijo GD	(1.5, 0.0)	converged	21	1.2057714	5.4763936e-07
Armijo GD	(0.5, 0.3)	converged	23	1.2057714	6.5266598e-07
Armijo GD	(1.0, 1.3)	converged	26	1.2057714	2.9687696e-07
Fixed GD	(1.5, 0.0)	converged	154	1.2057714	1.7848054e-06
Fixed GD	(0.5, 0.3)	converged	134	1.2057714	1.8070215e-06
Fixed GD	(1.0, 1.3)	converged	136	1.2057714	1.6793893e-06
Momentum	(1.5, 0.0)	converged	73	1.2057714	1.1150296e-06
Momentum	(0.5, 0.3)	converged	75	1.2057714	1.0694534e-06
Momentum	(1.0, 1.3)	converged	70	1.2057714	6.5263488e-07
Newton	(1.5, 0.0)	converged	7	1.2057714	6.7222603e-11
Newton	(0.5, 0.3)	converged	6	1.2057714	9.5132301e-11
Newton	(1.0, 1.3)	converged	5	1.2057714	3.0641839e-08

Applying other methods



Runtime Comparison Plots [see appendix for analysis]



Appendix

Optional part: Runtime comparisons

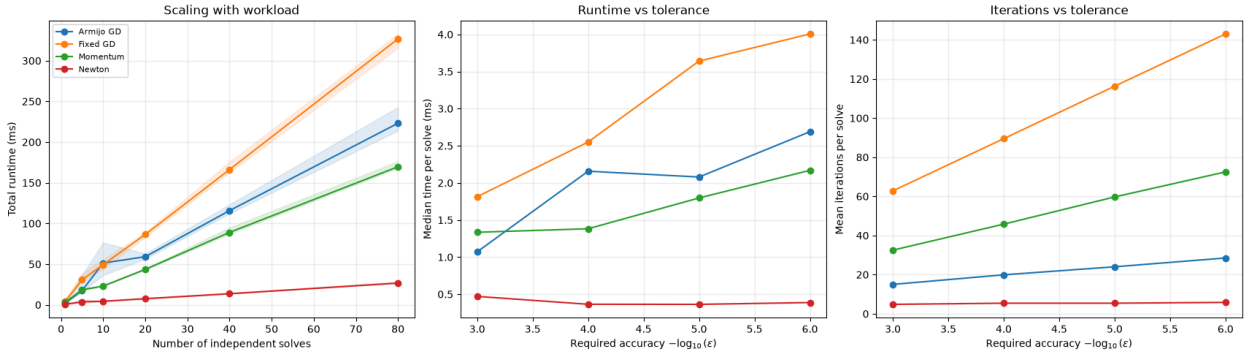
Here, workload refers to the number of starting points (n) that the algorithm needs to go through.

```
Median runtime (ms) for 80 independent solves, tolerance=1e-6:
   method  median_ms  mean_iterations
Armijo GD    223.141          27.475
 Fixed GD    326.863         139.162
 Momentum    169.671          69.487
   Newton     26.704           4.912

Median runtime per solve (ms), 10 starts, tolerance=1e-6:
   method  ms_per_solve  mean_iterations
Armijo GD         2.689           28.500
 Fixed GD         4.006          143.000
 Momentum         2.169           72.500
   Newton         0.388            5.800

Observed workload-scaling exponent from n = 10, 20, 40, 80:
Armijo GD : n^0.73
Fixed GD  : n^0.91
Momentum  : n^0.97
Newton    : n^0.86
```

Runtime plots: [added them here again for easier visualization]



Interpreting the graphs:

Plot 1 → Measures how much time it takes for a specific algorithm to optimize over the function given n starting points (workload).

Plot 2 → Measures how the run time of the algorithms changes as we increase the required accuracy (decreasing tolerance) using a negative logarithmic scale of base 10.

Plot 3 → Measures the number of iterations that a specific algorithm takes as we change (increase) the required precision (decreasing tolerance) using a negative logarithmic scale of base 10.

Brief conclusion: Newton's method is by far, the best method among these for this specific situation. My guess for why this happens is because the range of theta is only from 0 to $\frac{\pi}{2}$, so the non-polynomial behavior of the many $\sin(\theta)$ and $\cos(\theta)$ functions did not have enough room to appear so abruptly. Therefore, the quadratic approximation used in Newton's method is still good enough to quickly converge to the minimum.

Next, I would like to see how the conjugate gradient method performs here compared to Newton's method to see if it beats it.

AI Statement

I used ChatGPT to help me remember SageMath syntax and the methods to plot contour and surface plots. However, I still ended up needing to go back to SageMath's documentation because the original method for finding the solution for $\nabla A = 0$ was running forever when using $\text{solve}()$. Later, I ended up switching to $\text{find_root}()$ and it then worked! I also used it to convert some equations I had handwritten into KaTeX to add to my Notion. I also had GitHub copilot autosuggestions enabled when writing the code into VSCode. I incorporated some assert statements, print statements, and comments into the final Python Jupyter Notebook file (see "Links to Code Notebooks" appendix section below).

Link to conversation:

https://chatgpt.com/share/6ae12aaf-2378-83ea-998e-aca6cae04e97

Link to Handwritten Resolutions

https://drive.google.com/drive/folders/1pzPScT12W92K7cEk2xq8T6UAC8oWC1zm?usp=sharing

Links to Code Notebooks

SageMath Notebook (used for surface and contour plots, differentiation, and solving for $\nabla A = 0$):

https://colab.research.google.com/drive/1ZdVH3c5lDj6Hx9L-mR94pY6sBjDNbI6w?usp=sharing

Python Notebook (used to implement gradient descent methods and perform runtime comparisons):

https://colab.research.google.com/drive/1eRizQwo0BpxZ-2tlmPt-aty7BhUcx9×0?usp=sharing